Project PRG2782

Baobab Ridge Wildlife Rehabilitation Records System

A version-controlled C# Windows Forms application using text files, Git and GitHub

Group registration	At latest- 9th of October 2026
Submission deadline	23:59 of the 20th of October 2026 on BCConnect
Presentations	21st & 22nd of October 2026. Each group has 20 minutes to present, plus 5 minutes of questions.
Group size	3–4 students from the same class/module type (on campus or virtual)
Total marks	100

1. Scenario

Baobab Ridge Wildlife Rehabilitation Centre (a fictional centre) treats injured and orphaned wild animals and returns them to the wild once they are ready. Today the vets record each animal's progress on clipboards and in loose spreadsheets. Records go missing, get copied incorrectly, and nobody can see at a glance how many animals need intensive care and how many are ready for release.

Your team is the Centre's software unit. You have been asked to build a desktop application that records every animal in care, works out its **status** and **housing unit** from the vet's **Recovery Score**, and produces a summary report for the Centre manager. Because several developers will work on the same code, the project must be kept under version control on GitHub.

2. What you are building

- A **C# Windows Forms** application built in Visual Studio using .NET version 8.0 or newer.
- Data is stored in **plain text files only**. Databases and storage-related NuGet packages are not allowed.
- The project is managed with **Git** and hosted on **GitHub**.
- The application must support five features: **Add**, **View**, **Update**, **Delete** and **Summary Report**.

How to read this brief: "must" means it is a requirement and will be checked. "Should" means it is recommended. If something is not specified, you may decide for yourselves, but be ready to explain your decision at the presentation.

3. The animal record and validation rules

Every animal has the fields below. Every rule in the table must be enforced whenever the user adds or updates an animal. Text typed by the user is trimmed (leading and trailing spaces removed) before it is validated.

Field	Rules	Example
Animal ID	Required. The letters `WR-` followed by exactly 4 digits. The ID is converted to upper case before it is checked, so `wr-0007` is accepted and saved as `WR-0007`. Must be unique. Cannot be changed after the animal is saved.	`WR-0007`
Name	Required. 1–30 characters. Must not contain the pipe character `|`.	Naledi
Species	Required. 1–30 characters. Must not contain the pipe character `|`.	Serval

Field	Rules	Example
Age (years)	Required. A whole number from 0 to 100. Decimals and text are rejected.	3
Recovery Score	Required. A whole number from 0 to 100, entered by the vet after assessing the animal. Decimals and text are rejected.	62
Status	**Calculated** from the Recovery Score. The user never types this.	Recovering
Housing Unit	**Calculated** from the Recovery Score. The user never types this.	Outdoor Enclosure

4. Classification rules

The Recovery Score decides the animal's status and where it is housed. Scores are whole numbers, so there are no gaps between bands (for example, 19 is Critical and 20 is Serious).

Recovery Score	Status	Housing Unit	What it means
0–19	**Critical**	Intensive Care Unit	Needs 24-hour monitoring
20–39	**Serious**	High-Dependency Ward	Needs frequent checks through the day
40–59	**Stable**	Recovery Ward	Condition steady, daily checks
60–79	**Recovering**	Outdoor Enclosure	Rebuilding strength outdoors
80–100	**Release-Ready**	Pre-Release Camp	Being prepared for return to the wild

The status and housing unit must be spelled exactly as shown, because they are written to the data file and checked by assessors. The classification logic must live in **one method** that is called by both Add and Update. Do not copy it into two places.

5. File formats and locations

5.1 animals.txt

- One animal per line, with seven fields separated by the pipe character `|`, in this order: **ID | Name | Species | Age | Score | Status | Housing Unit**.
- No header line and no blank lines. Text encoding is UTF-8.
- The file is stored in the same folder as the running `.exe`.
- Your repository must include a starter `animals.txt` with **at least 8 valid records covering at least 4 different statuses**. Add it to the project with "Copy to Output Directory" set to "Copy if newer" so it lands beside the `.exe`.
- Assessors may replace your `animals.txt` with their own file in this format. Your application must load it without changes.
- When loading, a line that is **malformed** (wrong number of fields, non-numeric age or score, or an invalid ID) must be skipped, not crash the program. After loading, tell the user once how many lines were skipped. Skipped lines are not shown in the grid and do not need to be preserved when the file is next rewritten.

Example lines:

```
WR-0001|Thabo|African Penguin|4|85|Release-Ready|Pre-Release Camp
WR-0002|Naledi|Serval|3|62|Recovering|Outdoor Enclosure
WR-0004|Zola|Cape Vulture|12|19|Critical|Intensive Care Unit
```

5.2 summary.txt

Created in the same folder as the `.exe` each time the summary is generated. The file is **overwritten** each time, never appended to. It must use exactly this layout (the numbers below are examples):

```
BAOBAB RIDGE WILDLIFE REHABILITATION CENTRE
ANIMAL SUMMARY REPORT
Generated: 2026-10-14 09:32
----------------------------------------
Total animals:            12
Average age (years):      7.75
Average recovery score:   54.00
----------------------------------------
Animals per status:
  Critical:               2
  Serious:                2
  Stable:                 2
  Recovering:             3
  Release-Ready:          3
```

- The timestamp format is `yyyy-MM-dd HH:mm`.
- Averages are shown to **two decimal places using a full stop** as the decimal separator, regardless of the computer's regional settings.
- All five statuses are always listed, including those with a count of 0.
- If there are no animals, the count is 0 for everything and both averages are shown as `N/A`. The program must not crash.
- `summary.txt` is generated when the program runs. It does not need to be committed to Git.

6. Required features

Your application may use one form or several. Every feature below must be reachable from the user interface with clearly labelled controls.

Feature 1 – Add a new animal

1.1 The form has labelled input boxes for Animal ID, Name, Species, Age and Recovery Score, plus an **Add** button.

1.2 Clicking Add validates every field using the rules in Section 3. If any rule fails, **nothing is saved**, a message names the field and explains the problem (for example, "Recovery Score must be a whole number between 0 and 100."), and the cursor returns to that field.

1.3 If everything is valid, the program calculates the status and housing unit and **appends one new line** to `animals.txt`, leaving the existing lines untouched.

1.4 The user is shown the calculated status and housing unit (for example, in a confirmation message). The grid refreshes and the input boxes are cleared, ready for the next animal.

Feature 2 – View all animals

2.1 When the application starts, it loads `animals.txt` and shows every valid record in a **DataGridView**.

2.2 The columns appear in this order with these headers: **Animal ID, Name, Species, Age, Recovery Score, Status, Housing Unit**.

2.3 The grid is **read-only**: users cannot edit cells, add rows or delete rows directly in it. Clicking a cell selects the whole row. Records appear in the same order as in the file.

2.4 The grid refreshes automatically after every add, update and delete, without restarting the program and without duplicate rows.

2.5 If `animals.txt` does not exist, the program creates an empty one and shows an empty grid without an error.

Feature 3 – Update an animal

3.1 A record can be loaded for editing in **two ways**: by clicking a row in the grid, or by typing an Animal ID in a search box and clicking **Search** (exact match, not case-sensitive). If the ID is not found, the user is told and nothing else changes. Both methods load the record into the same input boxes.

3.2 While a record is loaded, the form is in **edit mode**: the Animal ID box is read-only, the Add button is disabled and a **Save Changes** button is enabled. A **Clear** button leaves edit mode and returns the form to Add mode with empty boxes.

3.3 Save Changes applies the same validation rules as Add (the ID is not re-checked for uniqueness because it cannot change). It recalculates the status and housing unit from the score, even if the score is unchanged.

3.4 Saving rewrites `animals.txt` so that **only the edited record changes**. No other record may be modified, lost or duplicated.

3.5 After a successful save, the grid refreshes and the form returns to Add mode.

Feature 4 – Delete an animal

4.1 The user selects a row in the grid and clicks **Delete**. If no row is selected, a message asks them to select one.

4.2 A **Yes/No confirmation** appears that shows the animal's ID and name, for example: "Delete WR-0007 (Naledi)? This cannot be undone." Choosing No changes nothing.

4.3 Choosing Yes removes that record from `animals.txt`. All other records stay exactly as they were. The grid refreshes, and if the deleted animal was loaded in the form, the form is cleared.

4.4 One record is deleted at a time.

Feature 5 – Generate a summary report

5.1 A **Generate Summary** button calculates the following from the data currently in `animals.txt`: total number of animals, average age, average recovery score, and the number of animals in each of the five statuses.

5.2 The results are displayed in labels or read-only text boxes on the form. A message box on its own is not enough.

5.3 The results are saved to `summary.txt` using the layout in Section 5.2.

5.4 The empty-data case described in Section 5.2 is handled without crashing.

7. Error handling and code documentation

- **File operations** (reading, appending, rewriting and writing the summary) must be wrapped in error handling. At a minimum, handle a missing file or folder, a file that is in use by another program (`IOException`) and permission problems (`UnauthorizedAccessException`). Show the user a friendly message, not a stack trace, and keep the program running. Test this by setting `animals.txt` to read-only and by opening it in another program.
- The program must **never crash with an unhandled exception** during normal use, including empty fields, letters in numeric boxes, an empty file, or clicking Delete/Save with nothing selected.
- Your code must include an **Animal class** (or record) and a **single classification method** as described in Section 4.
- Every method must have a comment above it explaining what it does. Any logic that is not obvious must have an inline comment. Delete old commented-out code before you submit.
- Use meaningful names for variables, methods, controls and forms (for example, `btnAdd`, not `button1`).

- The project must **build and run after a fresh clone** from GitHub, with no missing files and no file paths that only work on one computer (such as `C:\Users\...`).

8. Git and GitHub requirements

- **Start with Git.** Initialise the repository before writing any application code. The first commit contains only the blank Windows Forms project, a `.gitignore` for Visual Studio/.NET, and a `README.md`.
- **Branching.** The `main` branch is the stable branch and must always build. Develop each feature (Add, View, Update, Delete, Summary) on its own branch, for example `feature/add-animal`, and merge it into `main` through a **GitHub pull request**. Every group member must be responsible for at least one feature and have at least one merged pull request.
- **Commit volume.** Your history must contain at least **15 commits** in total. Each member must make at least **3 commits** under their own name (set `user.name` and `user.email` correctly), and commits must be made on at least **3 different days**.
- **Commit messages** must start with a verb and describe the change, for example "Add validation for animal ID format" or "Fix duplicate rows after update". Messages such as "update", "final", "changes" or "asdf" are not acceptable. Commit after each meaningful piece of work, not once per feature.
- **Keep the repository clean.** Do not commit `bin/`, `obj/`, `.vs/` or generated files such as `summary.txt`. Do not delete and recreate the repository, and do not force-push to rewrite history on `main`.
- **Push regularly**: at the end of every work session and always after finishing a feature. The code on GitHub must match the code you demonstrate.
- **README.md** must contain: project name, all member names and student numbers, how to build and run the program, a list of features, a short description of the two file formats, and any known limitations.
- **Access.** The repository must be public, or private with your lecturer's GitHub username added as a collaborator before the submission deadline.

9. Submission

Submit **one** GitHub repository URL, as well as a zip folder containing your entire project per group on BCConnect by 23:59 of the 20th of October 2026. Assessors will mark the latest commit on `main` made before the deadline. Later commits are ignored. Before you submit, tick every item:

- I cloned the repository into a new empty folder, built it and ran it, and all five features work.
- The starter `animals.txt` is in the repository and appears beside the `.exe` after a build.
- `README.md` is complete and every member has commits and a merged pull request.
- The repository is public, or the lecturer has been added as a collaborator.
- Everything is pushed and GitHub `main` matches my local `main`.

10. Group rules and academic integrity

- Groups have 3-4 students from the same class/module type. Register and let the lecturer know who the members of your group are, by the end of the first week of the course.
- The project mark is a **group mark**. The lecturer may adjust an individual's mark up or down based on evidence of contribution (commit history, pull requests and answers to questions).
- **Every student must present.** A student who does not present receives 0 for the project unless an absence has been formally approved by your lecturer.

- All code and documentation must be your group's own work. Copying from other groups or online sources, or submitting AI-generated code or text as your own, is academic misconduct and will be handled under Belgium Campus' integrity policy. This can result in a mark of 0 for the whole group. Official documentation and class notes may be used. If you adapt a short snippet from elsewhere, credit the source in a comment.

11. Presentation

Each group has **20 minutes to present**, followed by **5 minutes of questions**. A timekeeper will give a warning at 15 minutes and will stop you at 20 minutes. Anything you have not shown by then cannot be marked. You may not be presenting to your own lecturer. Virtual groups must share their screen and keep cameras on – you are expected to be dressed professionally in campus clothing.

Section	Time	What to cover
1. Introduction	3 min	The purpose of the project, who is on the team and which feature each member built, and the technologies used (C#, Windows Forms, Git, GitHub).
2. Live demonstration	8 min	Start with the application already open and your starter data loaded. Show, in this order: (a) the grid on startup; (b) adding an animal and the calculated status and housing unit, then one validation error; (c) updating an animal so its score crosses a band boundary (for example 79 → 80) and the status changes; (d) deleting an animal with the confirmation; (e) generating the summary and opening `summary.txt`; (f) one file error being handled gracefully.
3. Git and GitHub	4 min	Show the commit history, branches, pull requests and contributors on GitHub. Explain two or three key commits and one merge (or merge conflict) and how you handled it.
4. Code walkthrough	3 min	Walk through the classification method, how animals.txt is loaded and saved, and how errors are handled. At least two members must speak in this section.
5. Challenges and lessons	2 min	The biggest problems you hit, how you solved them, and what you would do differently.
Total	**20 min**	

- **Every member must present** a meaningful and roughly equal part, at least 3 minutes each.
- **Questions (5 minutes)** may be directed at any individual member and can cover any part of the code, the classification logic, file handling, error handling, and your use of Git and GitHub.
- Assessors may ask you to **run test values live**, for example adding an animal with a specific score, or changing a score by one point and predicting the result before you click the button.

12. How you will be assessed

Criterion	What assessors look for	Marks
1. Add animal	Form accepts all fields, validation rules enforced with clear messages, correct classification, record saved correctly to the file.	12
2. View animals	Data loaded from the file, grid readable with the correct columns, view stays up to date after changes, bad data handled.	8
3. Update animal	Record can be loaded by grid or ID search, validation applied, status and housing recalculated, only the edited record changes.	12
4. Delete animal	Row selection with confirmation, correct record removed from file and grid.	8

Criterion	What assessors look for	Marks
5. Summary report	Correct totals, averages and per-status counts, shown on the form and saved in the required format.	10
6. Git and GitHub	Meaningful and incremental commit history from every member, branches and pull requests, remote linked and pushed, lecturer access, README.	20
7. Error handling and code quality	File errors handled without crashing, sensible code structure, useful comments, builds from a fresh clone.	10
8. Presentation and Q&A	Clear structure within the time limit, evidence of teamwork, and individual understanding of the code shown through answers to questions and live tests.	20
Total		**100**